JS

NODE.JS ARCHITECTURE

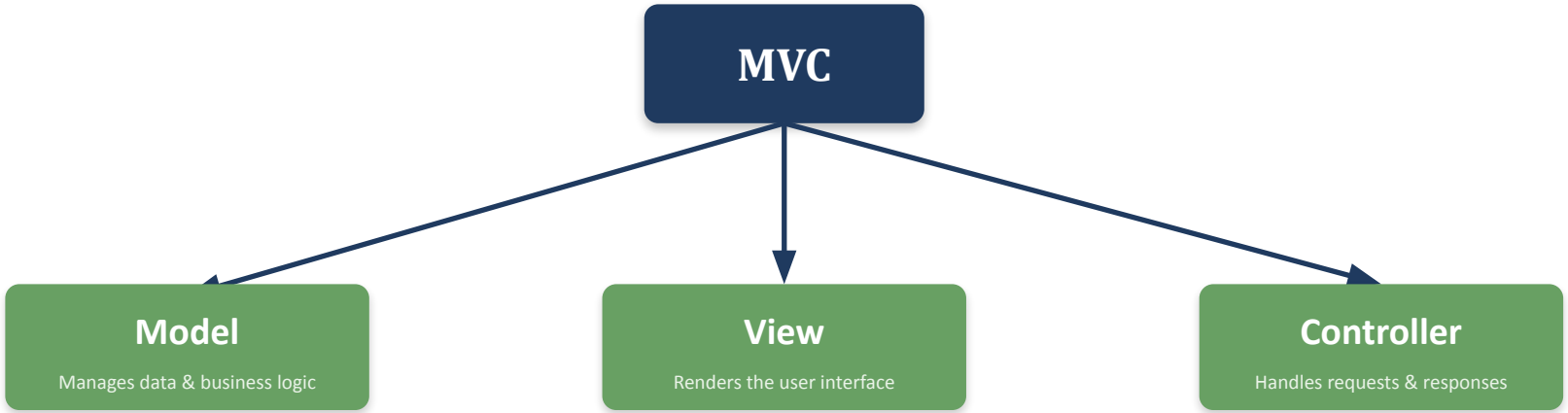
The MVC Pattern

Model – View – Controller: how modern web applications stay organized, scalable, and easy to maintain.

Model | **View** | **Controller**

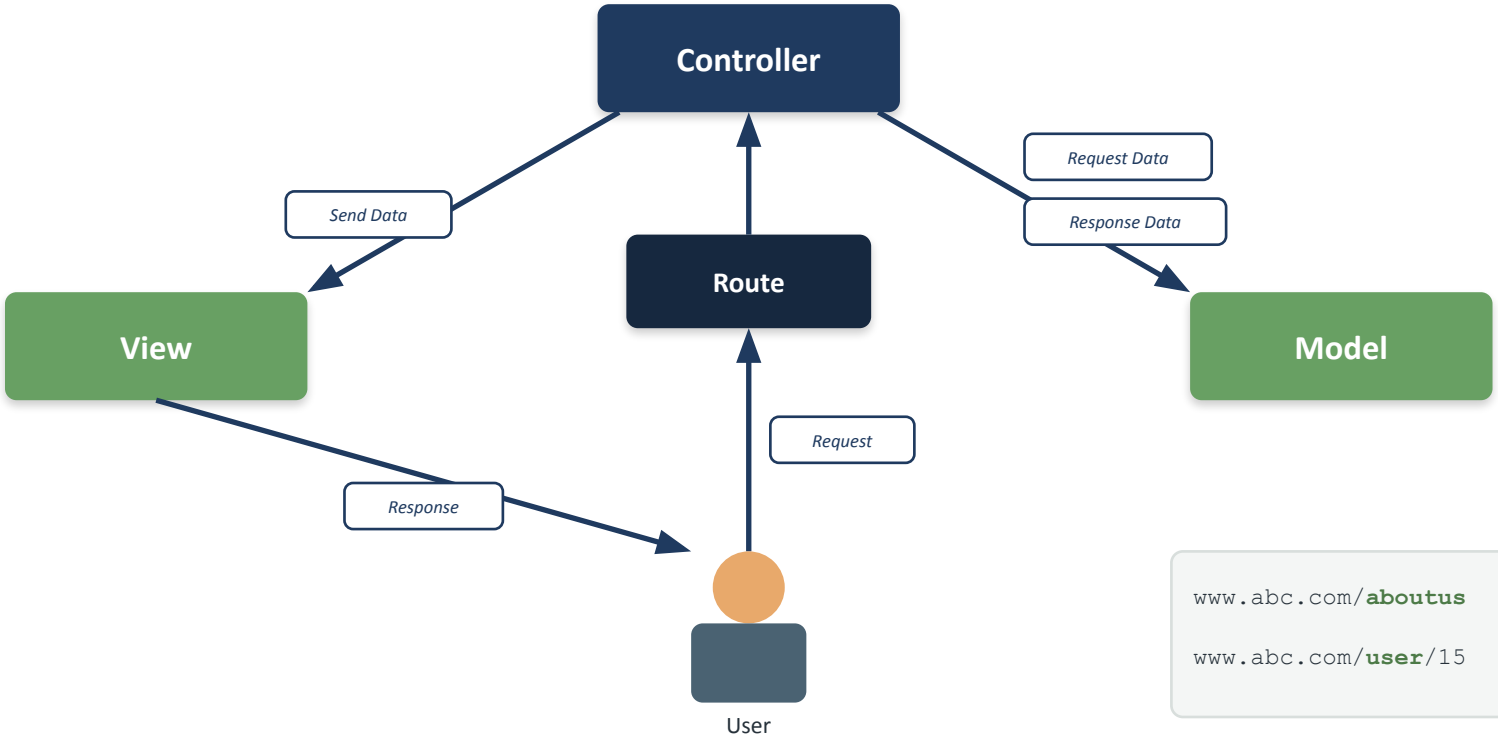
What is MVC Pattern?

MVC splits an application into three interconnected building blocks, each with a single responsibility.



Together, they separate data, presentation, and control flow — so each piece can change independently.

MVC Pattern Work Flow




```
www.abc.com/aboutus

www.abc.com/user/15
```

Benefits of MVC Pattern

1. **Organized Code**
   Clear separation keeps every file focused on one job

2. **Independent Blocks**
   Model, View, and Controller evolve without breaking each other

3. **Reduced Complexity**
   Large applications stay manageable as they grow

4. **Easy to Maintain**
   Bugs are isolated to a single, predictable layer

5. **Easy to Modify**
   Update the UI or logic without touching the rest

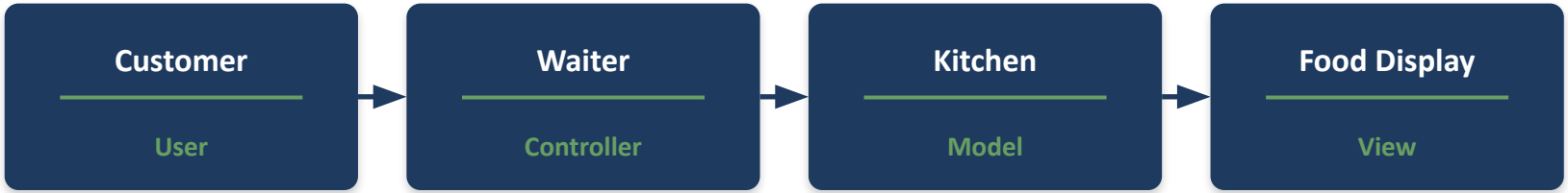
6. **Code Reusability**
   Models and controllers can be reused across views

7. **Improved Collaboration**
   Teams can work on separate layers in parallel

8. **Platform Independence**
   The same backend can serve web, mobile, or API clients

Real-Life Example: A Restaurant Analogy

A useful way to explain MVC to students — think of how a restaurant serves a meal.



1. **The Controller (waiter) receives the customer's request.**
   "I'd like the data for user number 10."
2. **The Controller asks the Model (kitchen) to prepare it.**
   "Get me user 10's data from the database."
3. **The Model fetches the data from the database.**
4. **The Controller passes the response to the View / front end.**

Life Without MVC: The Problem

Before introducing the pattern, show students the problem it solves. Suppose we build a simple API for users:

```
const express = require("express");

const app = express();

app.get("/users", async (req, res) => {
    const users = await User.find();
    res.json(users);
});

app.listen(5000);
```

This is simple — now imagine this file also has to handle:

- 20 routes
- 20 database queries
- Authentication
- Validation
- Error handling
- Business logic

Everything lands in one server.js file. It keeps growing — which is exactly why responsibilities need to be separated.

That separation is exactly what the MVC pattern provides.